

FAST NATIONAL UNIVERSITY
Department of Computer Science

SOFTWARE DESIGN SPECIFICATION

Smart Campus Application *An Integrated University Management System*

SUBMITTED BY

Name	Roll No.
Murtaza Hunaid Terai	24K-0782
Adnan Hatim	24K-0656

SUBMITTED TO

Mr. Syed Ahmed Khan
Software Design & Analysis | 4th Semester

Spring 2026

Table of Contents

Table of Contents.....	2
Chapter 1: Introduction.....	4
1.1 System Overview.....	4
1.2 Problem Statement.....	4
1.3 Scope.....	4
1.4 Objectives.....	5
1.5 Users and Stakeholders.....	5
1.6 System Requirements Summary.....	5
Functional Requirements.....	6
Non-Functional Requirements.....	6
1.7 Software Process Model.....	6
Chapter 2: System Analysis and Modeling.....	7
2.1 Use Case Diagram.....	7
2.1.1 Actors.....	7
2.1.2 Key Relationships.....	7
2.1.3 Use Case Diagram.....	7
2.1.4 Use Case Description Table — Key Use Cases.....	8
2.2 Class Diagram.....	10
2.2.1 Class Summary.....	10
2.2.2 OOP Relationships.....	10
2.2.3 Key Multiplicities.....	10
2.2.4 Class Diagram.....	11
2.2.5 BCE Model — Boundary, Control, Entity Classes.....	12
2.3 Activity Diagrams.....	15
2.3.1 QR Attendance Flow.....	15
2.3.2 Fee Payment Flow.....	16
2.3.3 Clearance Certificate Flow.....	18
2.3.4 Facility Booking Flow.....	19
2.3.5 Exam Result Flow.....	21
2.4 Sequence Diagrams.....	23
2.4.1 QR Attendance Sequence.....	23
2.4.2 Fee Payment Sequence.....	24

2.4.3 Clearance Certificate Sequence.....	25
2.4.4 Emergency Alert Sequence.....	26
2.5 Component Diagram.....	27
2.5.1 Component Summary.....	27
2.5.2 Interfaces and Dependencies.....	27
2.5.3 Component Diagram.....	28
2.6 Deployment Diagram.....	29
2.6.1 Network Zones.....	29
2.6.2 Key Artifacts and Technology Stack.....	29
2.6.3 Deployment Diagram.....	30
Chapter 3: Architectural View (4+1 Model).....	32
3.1 Logical View.....	32
3.2 Process View.....	32
3.3 Development View.....	33
3.4 Physical View.....	33
3.5 +1 Use Case View.....	34
Chapter 4: References.....	35

Chapter 1: Introduction

1.1 System Overview

The Smart Campus Application is a comprehensive, multi-role software platform designed to digitalize and streamline the core operations of a university campus. The system provides a unified, role-based interface for Students, Teachers, and Administrative Staff — including the Finance Department, Accounts Office, Registrar, Examination Department, and Security Personnel.

By consolidating fragmented, department-specific tools into a single integrated platform, the Smart Campus Application eliminates inefficiencies and enables real-time communication, automated workflows, and transparent access to institutional services for all stakeholders.

1.2 Problem Statement

Universities currently operate with disconnected, department-specific tools that create significant inefficiencies across all stakeholder groups. Students must navigate multiple portals for attendance, fees, exam results, and campus resources. Administrative departments rely on manual and siloed processes, causing delays, errors, and miscommunication.

Specific problems include:

- No centralized platform connecting students, faculty, and all administrative offices.
- Manual, error-prone attendance tracking with no real-time visibility.
- Students lack timely awareness of fee dues, deadlines, and payment status.
- Disconnected exam and result management with no digital rechecking workflow.
- No digital system for facility reservations or campus resource management.
- Clearance certificates require students to physically visit multiple offices.
- No unified emergency alert or campus security communication system.

1.3 Scope

The Smart Campus Application addresses the complete lifecycle of student-university interaction, covering:

- QR-code and GPS-based digital attendance tracking.
- Fee management with automated challan generation, payment processing, and scholarship application.
- Examination scheduling, grade publication, and rechecking workflows.
- Facility booking with admin approval and QR-based physical access control.
- Multi-department clearance certificate workflows.
- Campus-wide announcements, complaint management, and lost-and-found tracking.
- Security gate logging, visitor management, and emergency alert broadcasting.

The system is designed around object-oriented principles and supports role-based access for all user types. It does not include financial accounting, HR management, or external government reporting systems.

1.4 Objectives

1. Develop a centralized, multi-role platform accessible to all university stakeholders.
2. Implement QR-code and GPS-based digital attendance tracking.
3. Provide a real-time fee management module with automated challan generation and payment alerts.
4. Build a digital examination module for scheduling, results, and rechecking workflows.
5. Design a facility booking system with admin approval and QR-based access control.
6. Enable targeted campus-wide announcements and push notifications.
7. Create a complaint and lost-and-found system with smart item matching.
8. Implement a multi-department clearance certificate workflow.
9. Develop a security module with gate logs, visitor management, and emergency broadcasts.
10. Demonstrate OOP principles (inheritance, encapsulation, polymorphism, association) throughout the system design.
11. Produce complete and consistent UML documentation covering all required diagram types.

1.5 Users and Stakeholders

Role	Description	Key Responsibilities
Student	Enrolled university students	View timetable, pay fees, scan QR attendance, view results, book facilities, request clearance
Teacher	Faculty members	Generate QR codes, mark attendance, submit grades, post announcements
Finance Dept.	Financial administration staff	Generate fee challans, track payments, manage scholarships, send reminders
Accounts Office	Accounts verification staff	Verify payments, approve clearances, view audit logs
Exam Dept.	Examination administration	Schedule exams, publish results, process rechecking, issue transcripts
Registrar	Enrollment and registration office	Manage enrollment, approve course registration, issue certificates
Security Staff	Campus security personnel	Log gate entry/exit, register visitors, trigger emergency alerts
Admin	System administrator	Manage all users, approve bookings, view reports, configure system

1.6 System Requirements Summary

Functional Requirements

- User authentication with role-based access control.
- QR code generation, scanning, and GPS-based attendance verification.
- Fee challan generation, online payment processing, and receipt issuance.
- Exam scheduling, grade submission, result publication, and recheck processing.
- Facility availability checking, booking, admin approval, and QR access generation.
- Multi-department parallel clearance certificate workflow.
- Emergency alert broadcasting with campus zone mapping and all-clear notification.
- Complaint submission, tracking, and auto-escalation after 72 hours.
- Lost-and-found item reporting and smart matching.

Non-Functional Requirements

- Security: HTTPS, JWT authentication, role-based authorization, AES-256 data encryption at rest.
- Performance: API response time under 2 seconds for 95% of requests; emergency alert broadcast within 7 seconds.
- Availability: 99.9% uptime SLA with hot-standby database failover.
- Scalability: Horizontal scaling via load balancer across multiple application server instances.
- Maintainability: Modular microservice-inspired architecture with Docker containerization.

1.7 Software Process Model

The Smart Campus Application adopts the Unified Process (UP) model, an iterative and use-case-driven approach to software development. UP was chosen for this project for the following reasons:

- Use-case driven: The system has well-defined actors and interactions, making use-case modeling a natural fit for driving design and implementation.
- Architecture-centric: UP places early emphasis on establishing a stable software architecture, which is critical for a multi-module, multi-role system of this complexity.
- Iterative risk reduction: High-risk components such as QR-based attendance verification, payment gateway integration, and emergency alert broadcasting can be addressed in early iterations, reducing overall project risk.
- Supports evolving requirements: The iterative nature of UP accommodates feedback from stakeholders across multiple disciplines (Finance, Registrar, Security) without requiring a complete restart of the design process.

The project proceeds through four UP phases: Inception (scope definition and use case identification), Elaboration (architectural design and high-risk use case implementation), Construction (full feature build-out across all modules), and Transition (testing, deployment, and handover).

Chapter 2: System Analysis and Modeling

This chapter presents the complete UML design artifacts for the Smart Campus Application. All diagrams were developed in accordance with UML 2.x standards and cover the structural, behavioral, and interaction aspects of the system.

2.1 Use Case Diagram

The Use Case Diagram identifies all system actors and their interactions with the system's functional use cases. The diagram models 8 actors and spans 9 functional categories within the Smart Campus Application system boundary.

2.1.1 Actors

Actor	Type	Description
Student	Primary	Enrolled student interacting with academic, financial, and campus services
Teacher	Primary	Faculty member managing attendance, grades, and announcements
Finance Dept.	Primary	Generates challans, manages scholarships, tracks payments
Accounts Office	Primary	Verifies payments, approves clearances, maintains audit logs
Exam Dept.	Primary	Schedules exams, publishes results, processes rechecking
Registrar	Primary	Manages enrollment, approves registrations, issues certificates
Security Staff	Primary	Manages gate access, registers visitors, triggers alerts
Admin	Primary	Oversees the entire system, approves bookings, views reports

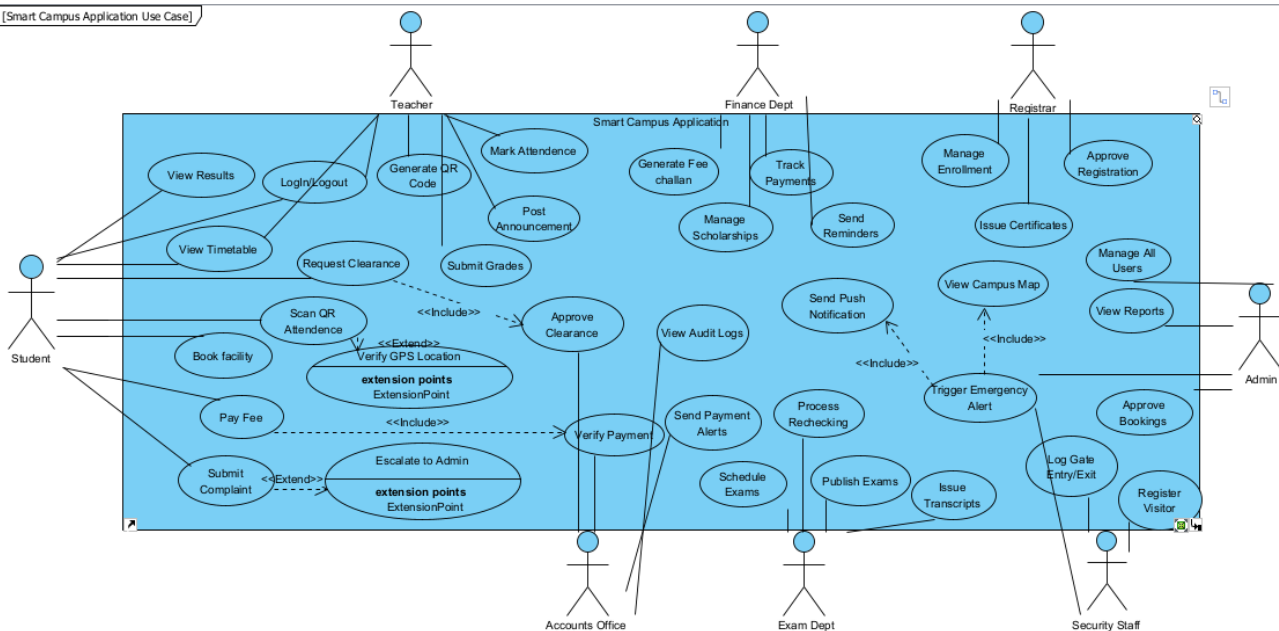
2.1.2 Key Relationships

- Pay Fee <<include>> Verify Payment — every payment transaction requires payment verification.
- Request Clearance <<include>> Approve Clearance — clearance always requires multi-department approval.
- Scan QR Attendance <<extend>> Verify GPS Location — GPS check is an optional extension triggered when attendance is scanned.
- Submit Complaint <<extend>> Escalate to Admin — escalation is triggered only if complaint remains unresolved after 72 hours.

2.1.3 Use Case Diagram

The Use Case Diagram was developed using Sparx Enterprise Architect and is presented below. All 8 actors, use cases across 9 functional sections, and all include/extend relationships are shown within the system boundary.

uc [Smart Campus Application Use Case]



2.1.4 Use Case Description Table — Key Use Cases

UC-01: Scan QR Attendance

Use Case ID	UC-01
Name	Scan QR Attendance
Actor(s)	Student, Teacher, System
Precondition	Teacher has started class; Student is enrolled in the course; QR code is active and not expired.
Main Flow	1. Teacher starts class and generates a QR code. 2. System stores QR with a 10-minute expiry. 3. Student opens app and scans the QR code. 4. System validates QR and verifies student GPS location. 5. System marks attendance as PRESENT. 6. System checks attendance threshold; sends low-attendance alert if below 75%.
Alternative Flow	If QR is expired or GPS verification fails, system returns an error and attendance is not recorded.
Postcondition	Attendance record updated in the database; student receives confirmation notification.

UC-02: Pay Fee

Use Case ID	UC-02
Name	Pay Fee
Actor(s)	Student, Finance Dept., Accounts Office, System
Precondition	Fee challan has been generated by Finance Dept.; Student is logged in.
Main Flow	1. Student views fee challan in the app. 2. Student initiates payment and selects method. 3. System processes payment via payment gateway. 4. Accounts Office verifies transaction. 5. System generates receipt and sends push notification to student.

Alternative Flow	If payment fails, student may retry up to 3 times. Finance sends reminder notifications for overdue challans.
Postcondition	Fee status updated to PAID; receipt stored and sent to student; audit log entry created.

UC-03: Request Clearance Certificate

Use Case ID	UC-03
Name	Request Clearance Certificate
Actor(s)	Student, ClearanceController, Finance, Library, Registrar, Accounts Office
Precondition	Student is enrolled; all dues must be cleared for approval to succeed.
Main Flow	1. Student submits clearance request. 2. System checks eligibility. 3. System sends parallel approval requests to Finance, Library, and Registrar. 4. All departments review and approve. 5. Accounts Office generates and issues the clearance certificate.
Alternative Flow	If any department rejects, student receives a PENDING notice listing outstanding departments.
Postcondition	Clearance certificate issued and downloadable from student portal.

UC-04: Trigger Emergency Alert

Use Case ID	UC-04
Name	Trigger Emergency Alert
Actor(s)	Security Staff, Admin, AlertController, NotificationBroadcaster, All Users
Precondition	Security staff is authenticated and on duty.
Main Flow	1. Security staff triggers alert with type and affected zone. 2. AlertController creates and logs the alert. 3. CampusMapService returns affected zones and evacuation routes. 4. NotificationBroadcaster pushes alert to all users in affected zones. 5. Coverage summary returned to Security Staff.
Alternative Flow	Once incident is resolved, Security Staff triggers resolution; system broadcasts All Clear to all users.
Postcondition	All users in affected zones notified; alert logged in audit database; resolution broadcast if resolved.

2.2 Class Diagram

The Class Diagram models the complete structural architecture of the Smart Campus Application. It presents all domain classes, their attributes, operations, and the full set of relationships between them. Classes are organized into 8 packages: Users & Roles, Academic, Examination, Finance, Security, Communications, Facilities & Clearance, and Enumerations.

2.2.1 Class Summary

Package	Classes	Count
Users & Roles	User (abstract), Student, Teacher, StaffMember (abstract), Admin	5
Academic	Course, Department, Timetable, AttendanceRecord, QRCode	5
Examination	ExamSchedule, Result	2
Finance	FeeRecord, FeeLineItem, Payment, Scholarship	4
Security	SecurityStaff, GateLog, Visitor, EmergencyAlert	4
Communications	Notification, Announcement, Complaint, LostAndFound	4
Facilities & Clearance	FacilityBooking, Facility, Clearance, DeptApproval	4
Enumerations	RoleEnum, StatusEnum, AttendanceStatus, PaymentMethod, NotificationType, AlertType, BookingStatus, GradeEnum	8

2.2.2 OOP Relationships

- Inheritance: Student, Teacher, StaffMember extend User (abstract). Admin and SecurityStaff extend StaffMember.
- Association: Student enrolled in Course (many-to-many); Teacher teaches Course (one-to-many).
- Aggregation: Department contains many Courses (Department is a whole, Courses are independent parts).
- Composition: FeeRecord is composed of FeeLineItems; Clearance is composed of DeptApprovals (parts cannot exist without the whole).

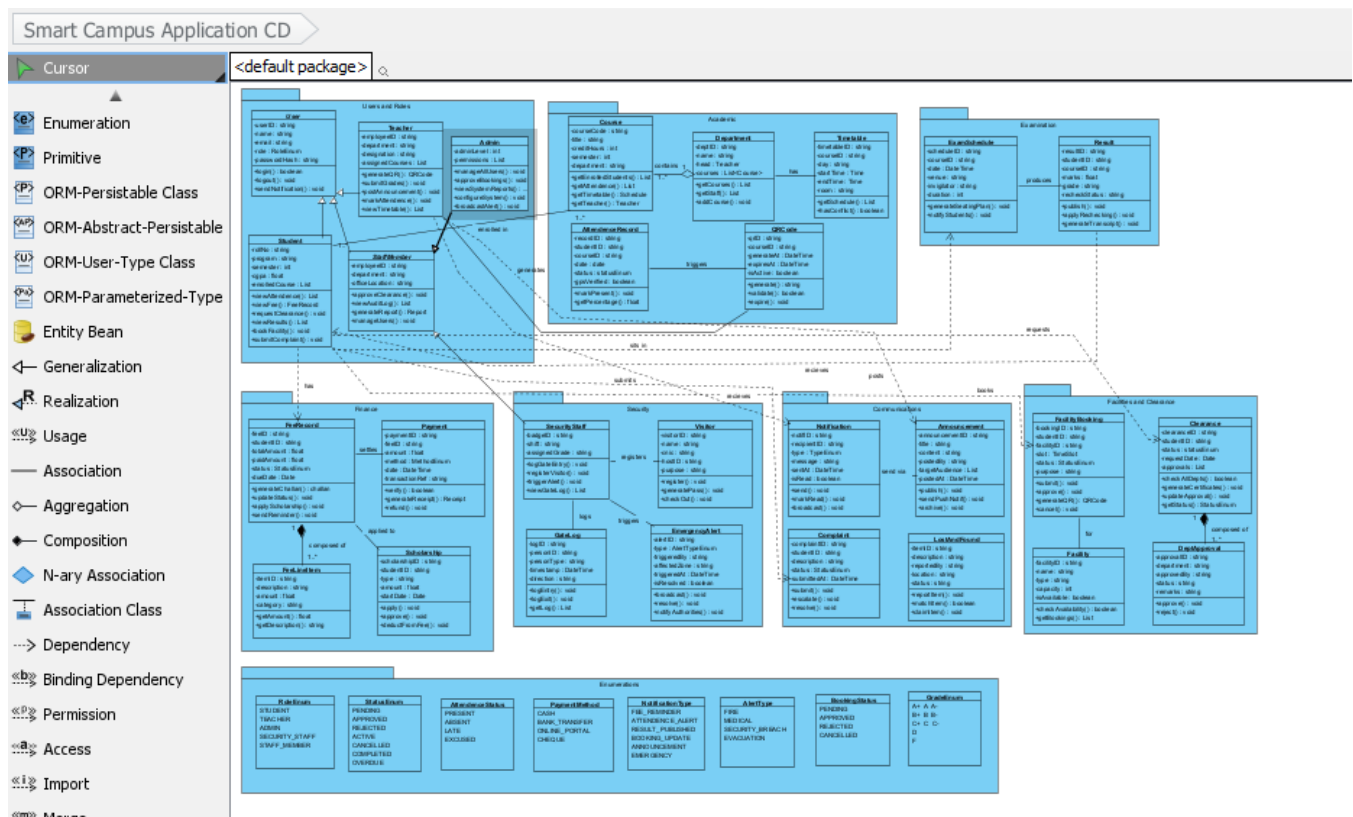
2.2.3 Key Multiplicities

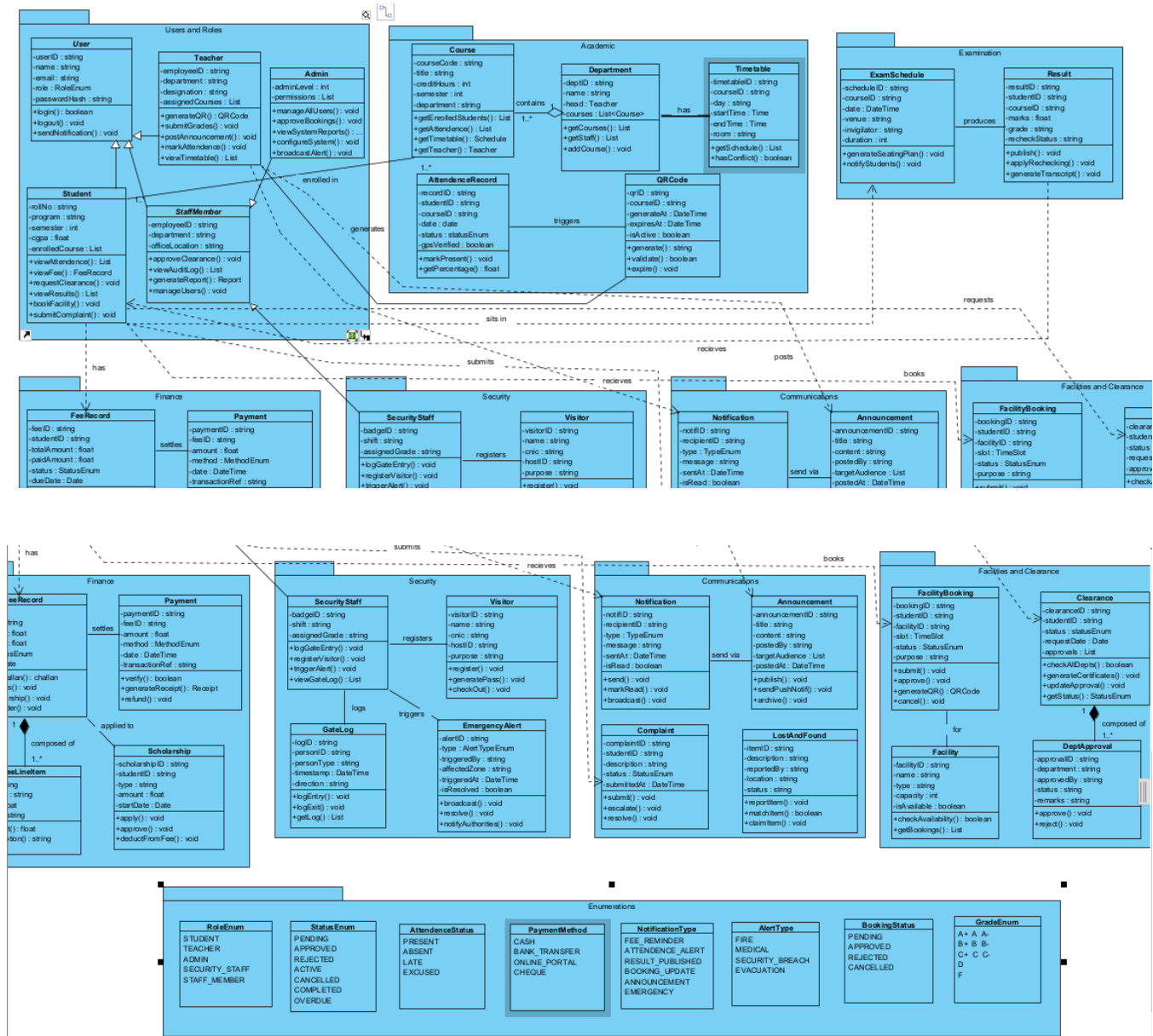
Relationship	Multiplicity	Type
Student — Course (enrolled in)	1..* to 1..*	Association
Teacher — Course (teaches)	1 to 1..*	Association

Relationship	Multiplicity	Type
Department — Course (contains)	1 to 1..*	Aggregation
FeeRecord — FeeLineItem	1 to 1..*	Composition
Clearance — DeptApproval	1 to 1..*	Composition
Payment — FeeRecord (settles)	0..* to 1	Association
Student — FeeRecord (has)	1 to 1	Association
SecurityStaff — GateLog (logs)	1 to 0..*	Association
SecurityStaff — Visitor (registers)	1 to 0..*	Association
FacilityBooking — Facility (for)	0..* to 1	Association

2.2.4 Class Diagram

The Class Diagram was designed using Visual Paradigm with all 8 packages and complete enumeration classes.





2.2.5 BCE Model — Boundary, Control, Entity Classes

In accordance with the Boundary-Control-Entity (BCE) architectural pattern, all system classes are classified into one of three stereotypes:

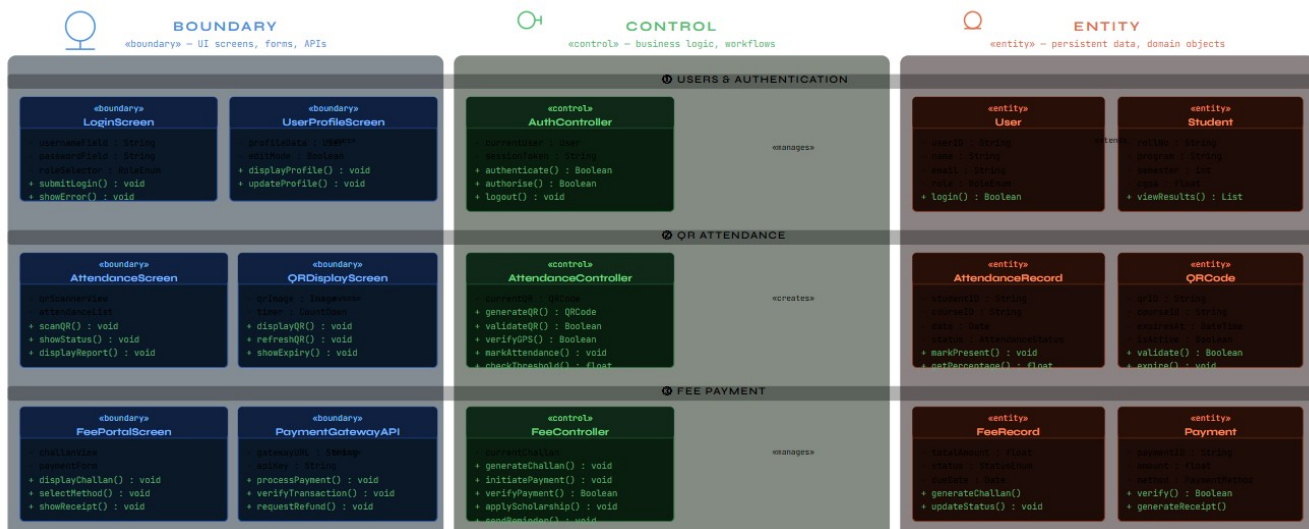
- Boundary classes interact with external actors (UI screens, API interfaces).
- Control classes encapsulate business logic and orchestrate workflows between boundary and entity classes.
- Entity classes represent persistent domain objects stored in the database.

The BCE interaction rule mandates: Actors communicate only with Boundary classes; Boundary classes communicate with Control classes; Control classes read and write Entity classes. Actors must never access Entity classes directly.

Stereotype	Classes	Count
«boundary»	LoginScreen, UserProfileScreen, AttendanceScreen, QRDisplayScreen, FeePortalScreen, PaymentGatewayAPI, ExamPortalScreen, TranscriptAPI, FacilityBookingScreen, AdminApprovalScreen, ClearanceRequestScreen, DeptApprovalScreen, NotificationScreen, AnnouncementScreen, SecurityDashboard, EmergencyAlertScreen, ComplaintFormScreen, LostFoundScreen, CourseManagementScreen, TimetableScreen	20
«control»	AuthController, AttendanceController, FeeController, ExamController, BookingController, ClearanceController, NotificationController, AlertController, ComplaintController, CourseController	10
«entity»	User, Student, Teacher, Admin, SecurityStaff, AttendanceRecord, QRCode, FeeRecord, FeeLineItem, Payment, Scholarship, ExamSchedule, Result, FacilityBooking, Facility, Clearance, DeptApproval, Notification, Announcement, EmergencyAlert, GateLog, Visitor, Complaint, LostAndFound, Course, Department, Timetable, Report	28

BCE MODEL — BOUNDARY · CONTROL · ENTITY CLASSES

Smart Campus Application · SDA Project · FAST University · 24K-0782 · 24K-0656





2.3 Activity Diagrams

Activity Diagrams model the major workflows of the Smart Campus Application. Each diagram uses UML swimlanes to assign responsibilities to specific actors, decision diamonds for branching logic, and fork/join bars for concurrent (parallel) activities.

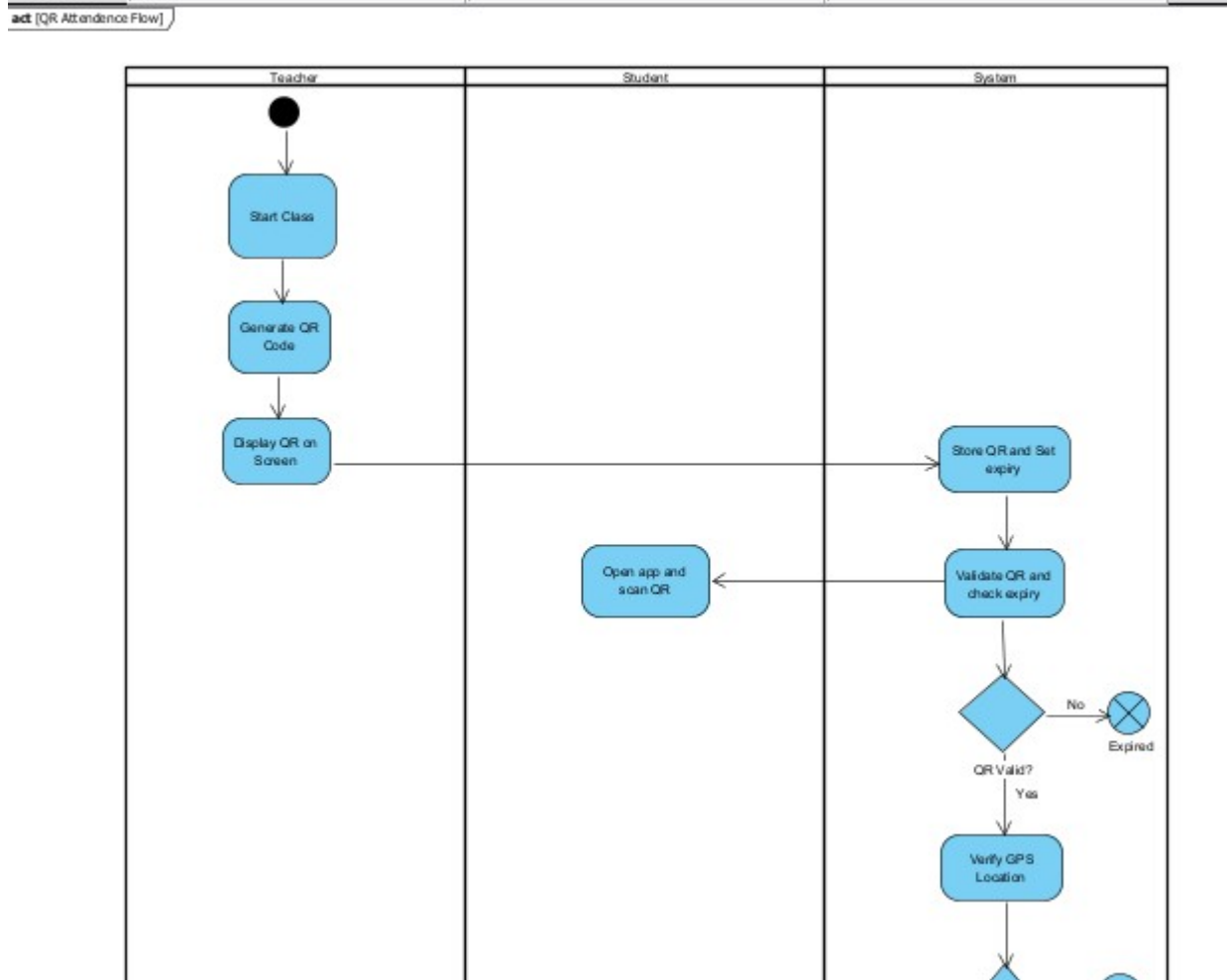
Five key workflows are modeled:

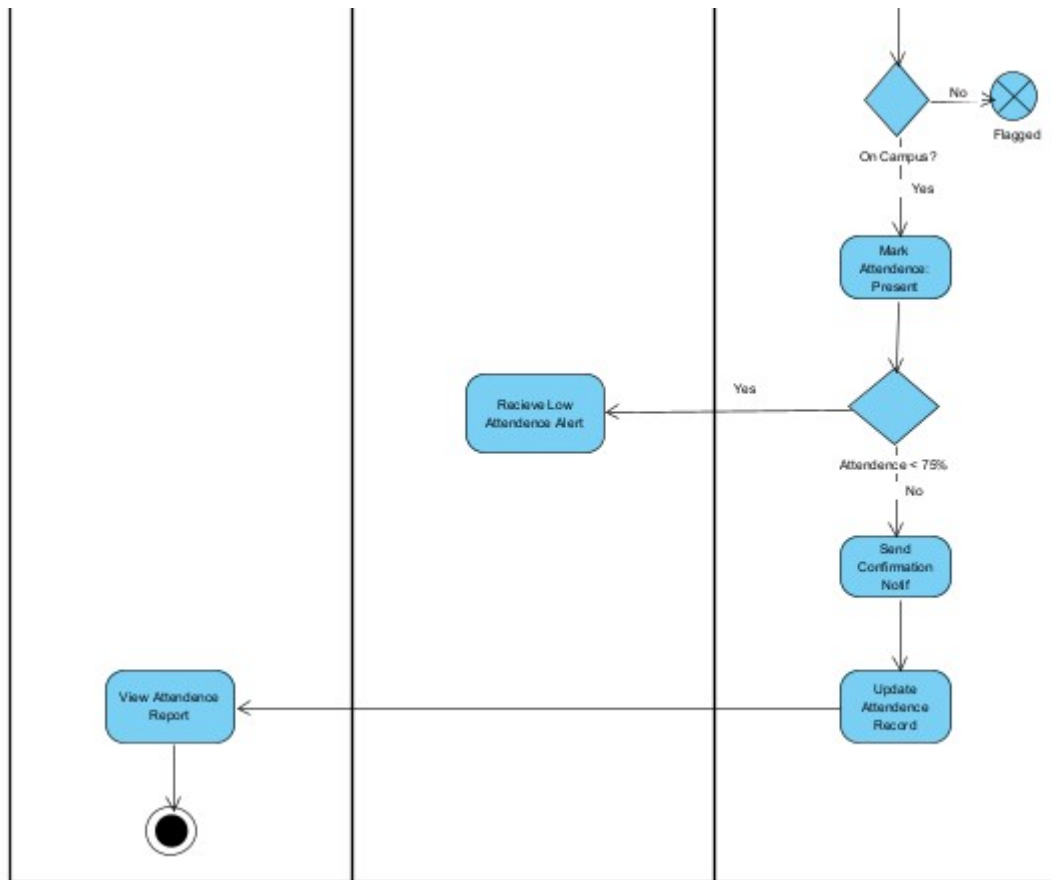
2.3.1 QR Attendance Flow

Swimlanes: Teacher | System | Student

This workflow covers the complete QR-code-based attendance process: the teacher starts class and generates a time-limited QR code; the system stores it with a 10-minute expiry; the student scans the QR via the mobile app; the system validates the QR and verifies the student's GPS location; attendance is marked PRESENT if both checks pass; a low-attendance alert is sent if the student's cumulative attendance falls below 75%.

Key decisions: QR valid? (Yes → GPS verify | No → error); On campus? (Yes → mark present | No → flagged).

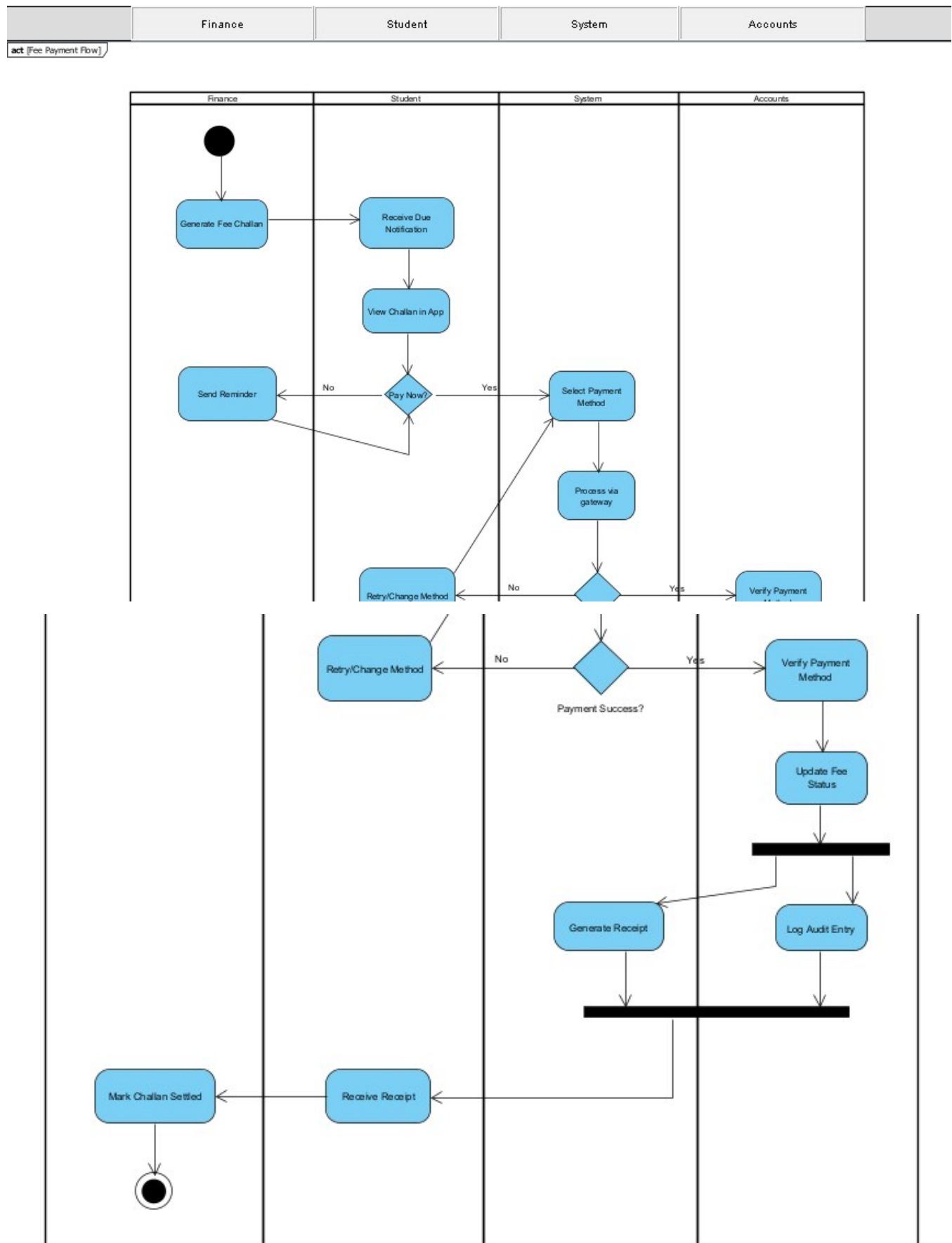




2.3.2 Fee Payment Flow

Swimlanes: Finance | Student | System | Accounts

Finance generates a fee challan and the student is notified. The student reviews the challan and initiates payment. The system processes the payment through the payment gateway. On success, Accounts verifies the transaction. A parallel fork generates the receipt and logs an audit entry simultaneously. If payment fails, the student may retry up to 3 times (LOOP fragment). Finance marks the challan as settled.

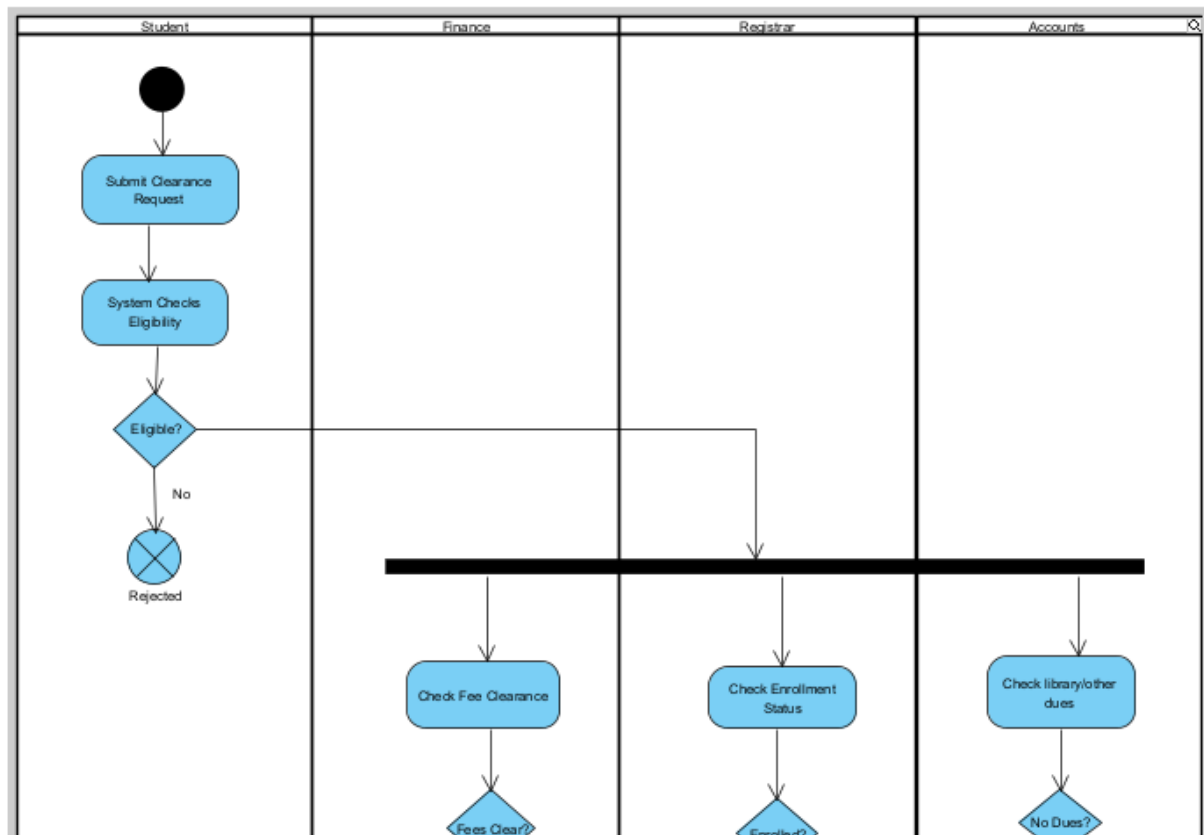


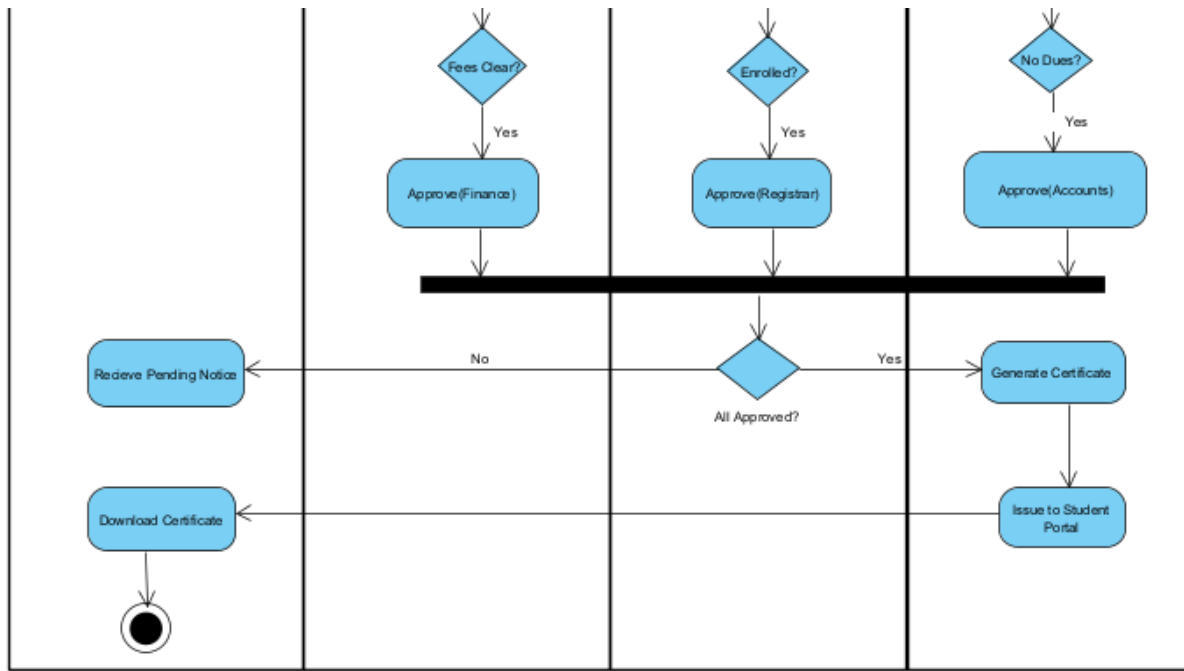
2.3.3 Clearance Certificate Flow

Swimlanes: Student | Finance | Registrar | Accounts

The student submits a clearance request. The system checks eligibility. A parallel fork (PAR fragment) sends simultaneous check requests to Finance (fee status), Registrar (enrollment status), and Accounts (library and other dues). A join bar waits for all three approvals before proceeding. If all are approved, Accounts generates and issues the certificate. If any department is pending, the student receives a pending notice.

act [Clearance Certificate Flow]

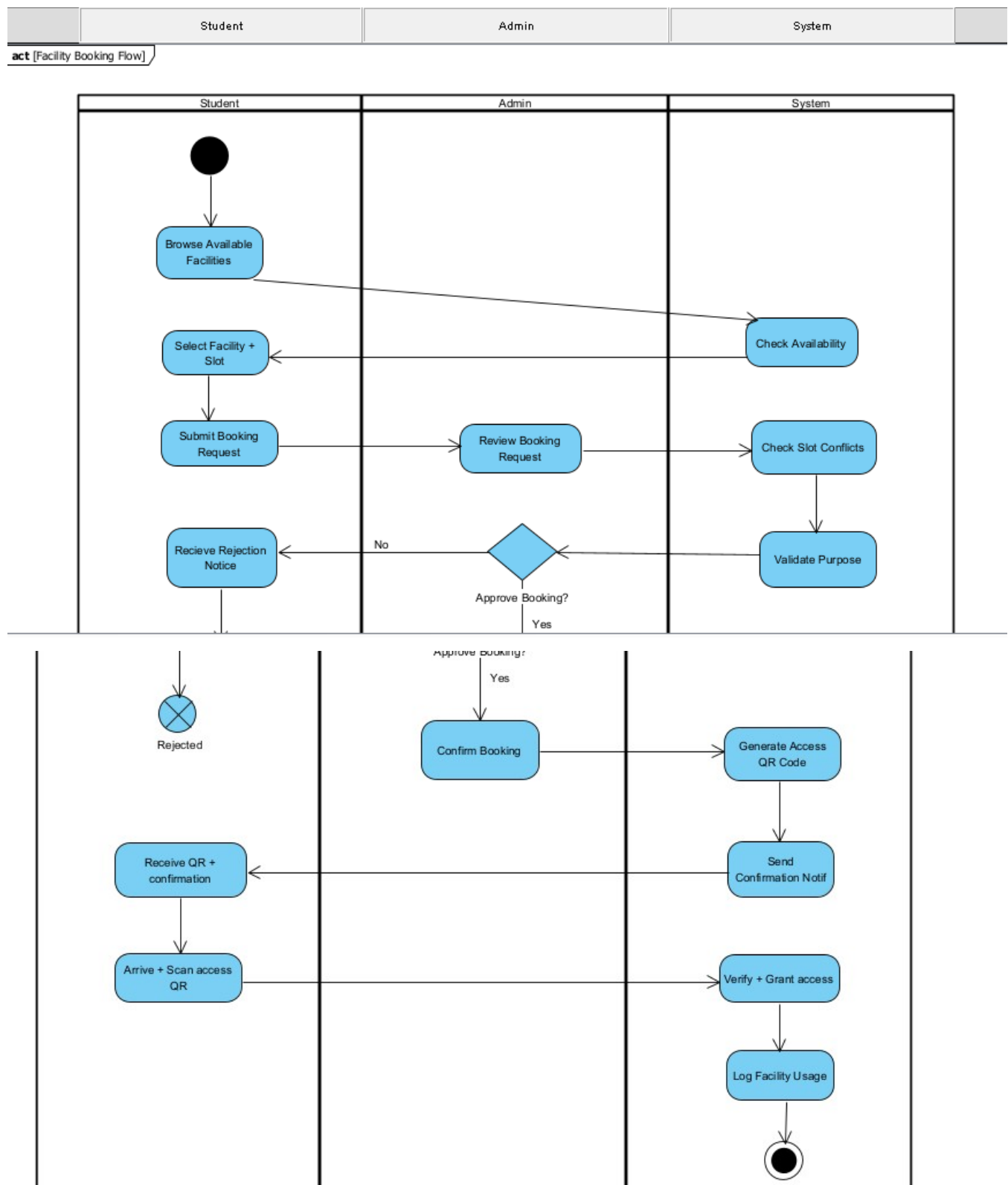




2.3.4 Facility Booking Flow

Swimlanes: Student | Admin | System

The student browses available facilities. The system returns availability data. The student selects a slot and submits a booking request. Admin reviews the request (with the System checking slot conflicts in parallel). Admin approves or rejects. On approval, the system generates a QR access code and sends a confirmation to the student. At booking time, the student scans the QR for physical access; the system verifies and logs facility usage.

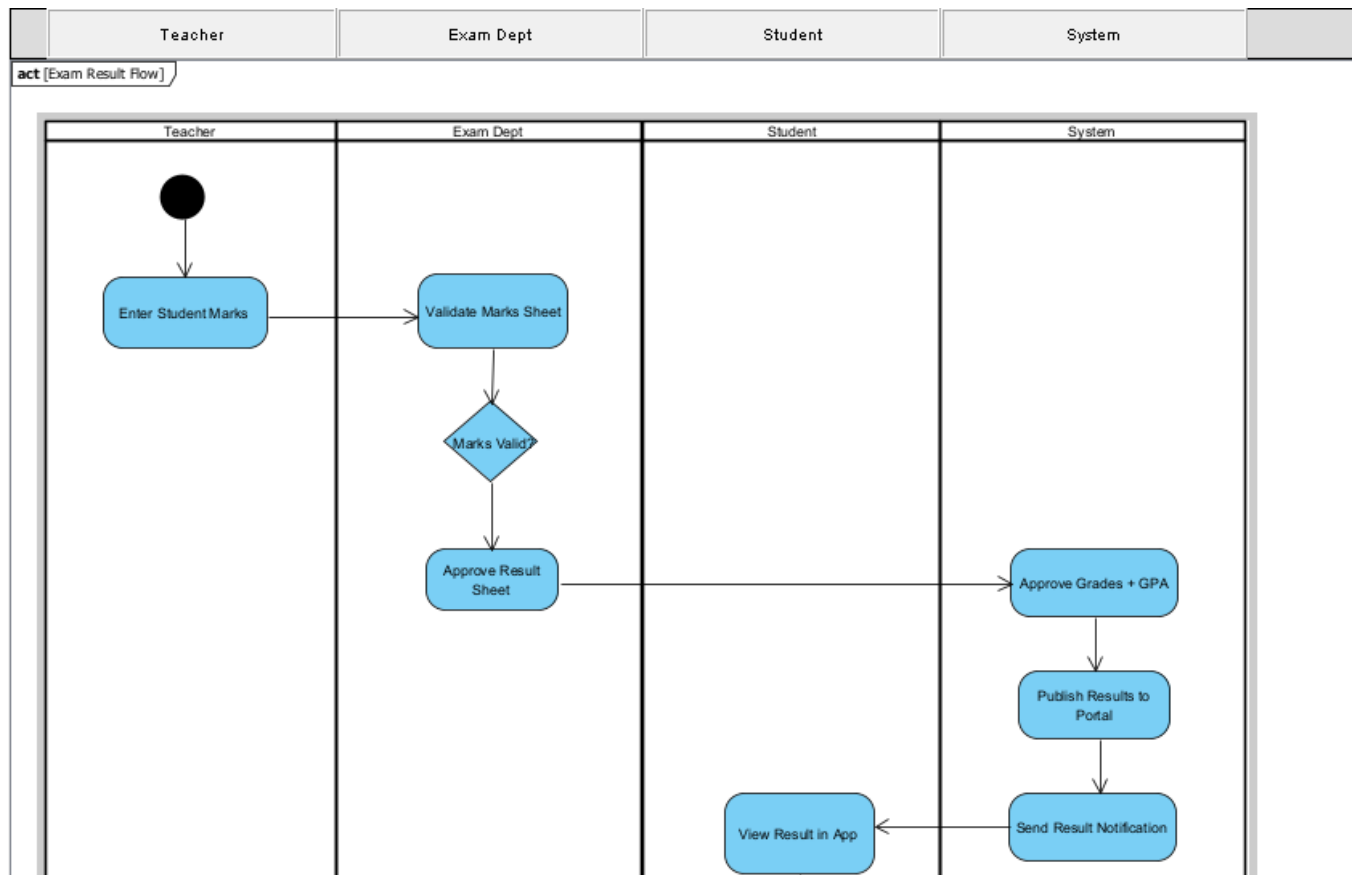


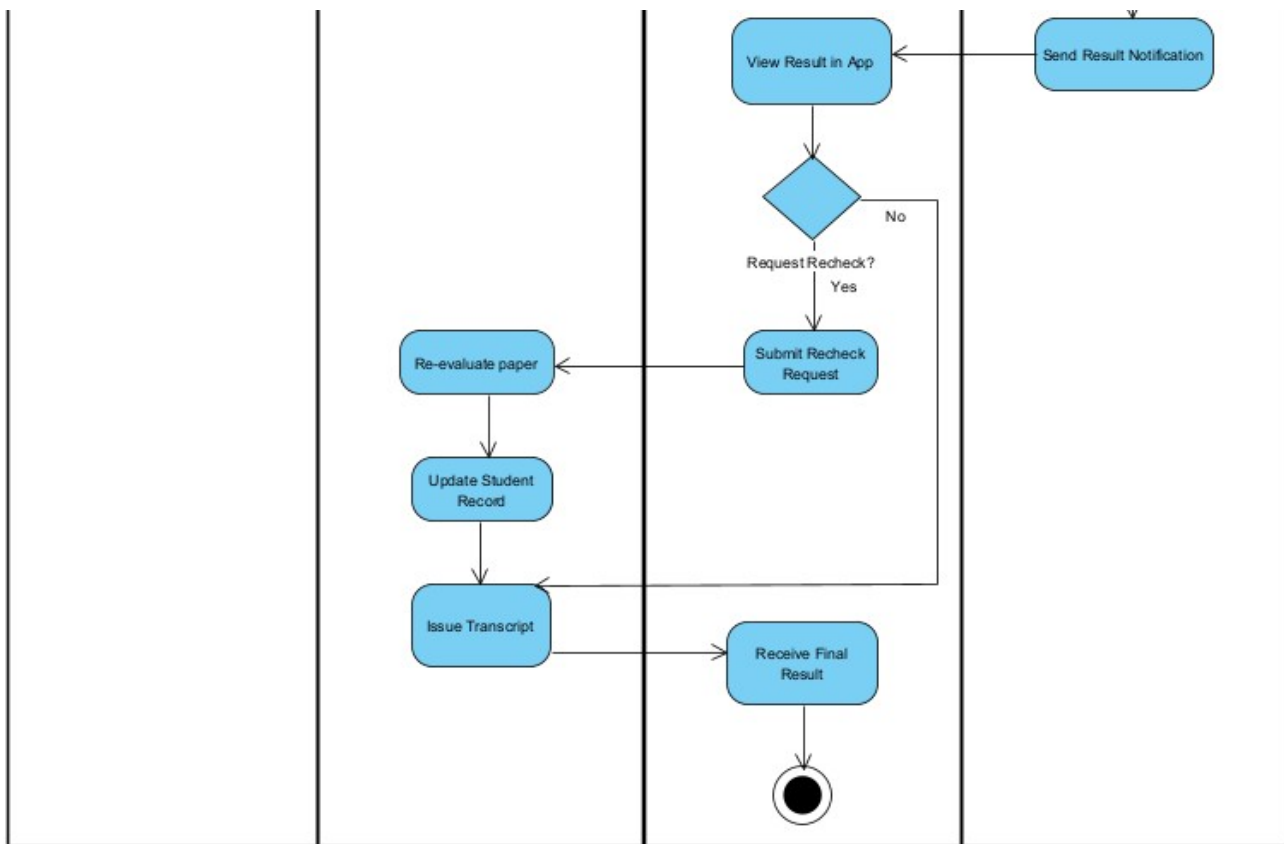
2.3.5 Exam Result Flow

Swimlanes: Teacher | Exam Dept. | Student | System

The teacher enters student marks. The Exam Department validates the marks sheet; if invalid, the teacher must correct and resubmit (LOOP). On approval, the system computes grades and GPA, then publishes results to the student portal. Students are notified. A student may optionally request rechecking; the Exam Dept. re-evaluates and updates the grade if changed. The Exam Dept. issues the transcript and updates the student record.

Additionally, a Master Activity Diagram combines all five flows into a single diagram triggered from a main menu decision node, with a looping mechanism allowing users to return to the menu and select another feature, and terminating at logout.





2.4 Sequence Diagrams

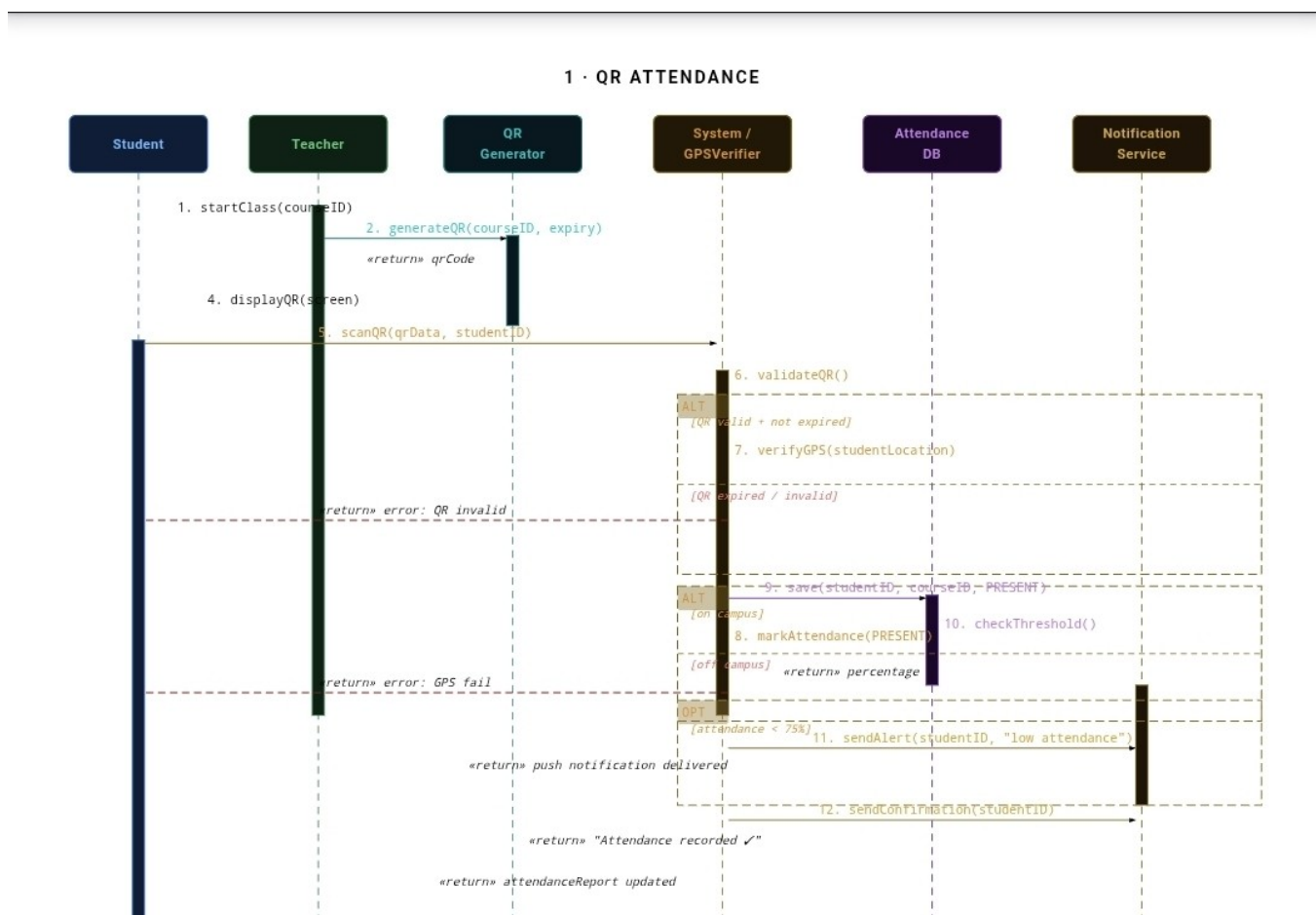
Sequence Diagrams illustrate the time-ordered interaction between objects for key use cases. Each diagram shows lifelines, activation boxes, synchronous messages, return messages, and combined fragments (ALT, LOOP, OPT, PAR) to model conditional and concurrent behavior.

Four use cases are modeled with sequence diagrams:

2.4.1 QR Attendance Sequence

Objects: Student | Teacher | QRGenerator | System/GPSVerifier | AttendanceDB | NotificationService

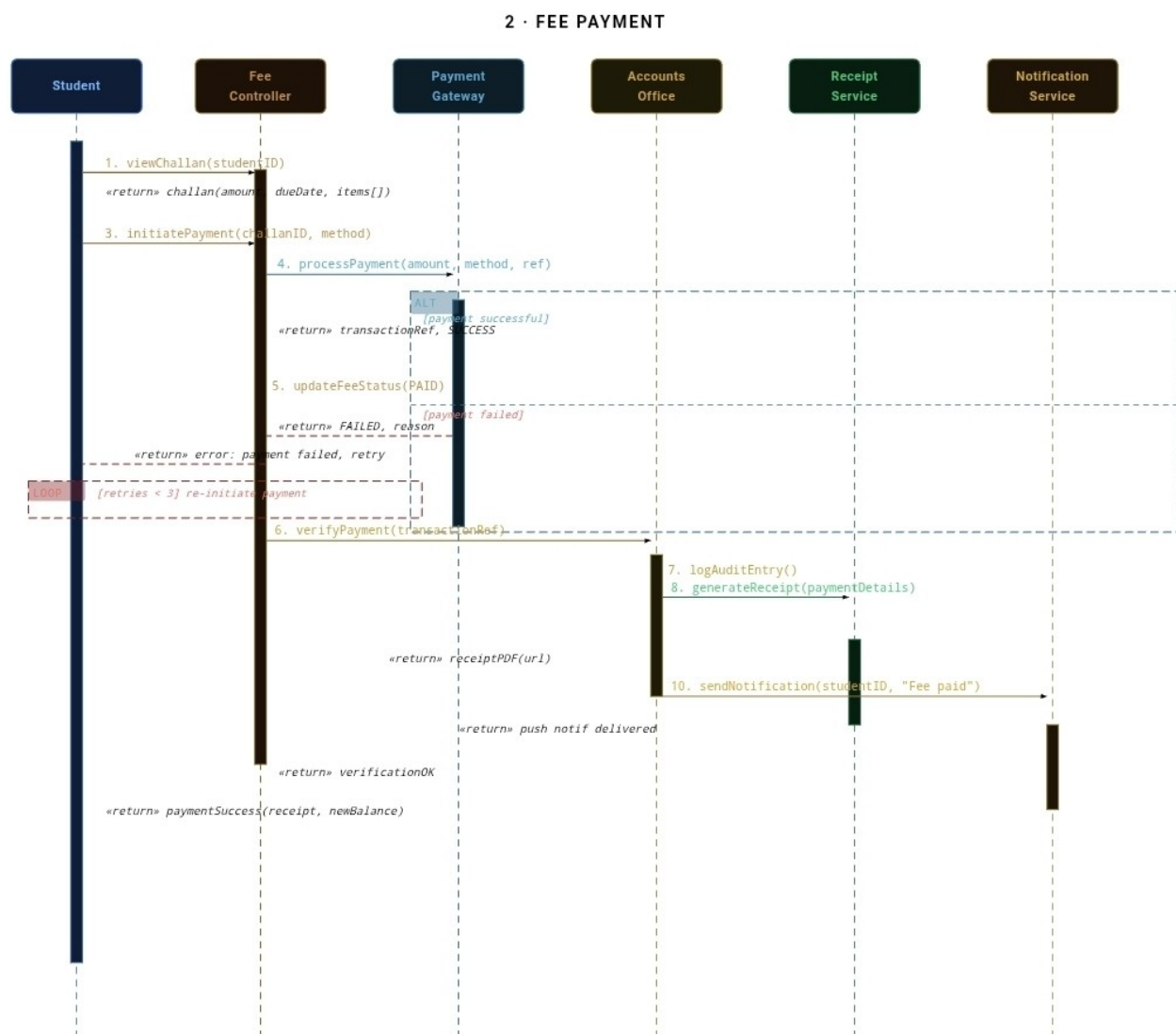
The Teacher calls `startClass(courseID)` and `generateQR(courseID, expiry)` on the QRGenerator. The QRGenerator returns a `qrCode`. The Teacher displays the QR. The Student scans and calls `scanQR(qrData, studentID)` on the System. The System calls `validateQR()` — an ALT fragment branches on valid/invalid QR. On validity, `verifyGPS()` is called — a second ALT fragment branches on campus/off-campus. On success, `save(studentID, courseID, PRESENT)` is called on AttendanceDB, which calls `checkThreshold()`. An OPT fragment triggers `sendAlert()` via NotificationService if `attendance < 75%`. Finally, the System sends a confirmation notification to the Student.



2.4.2 Fee Payment Sequence

Objects: Student | FeeController | PaymentGateway | AccountsOffice | ReceiptService | NotificationService

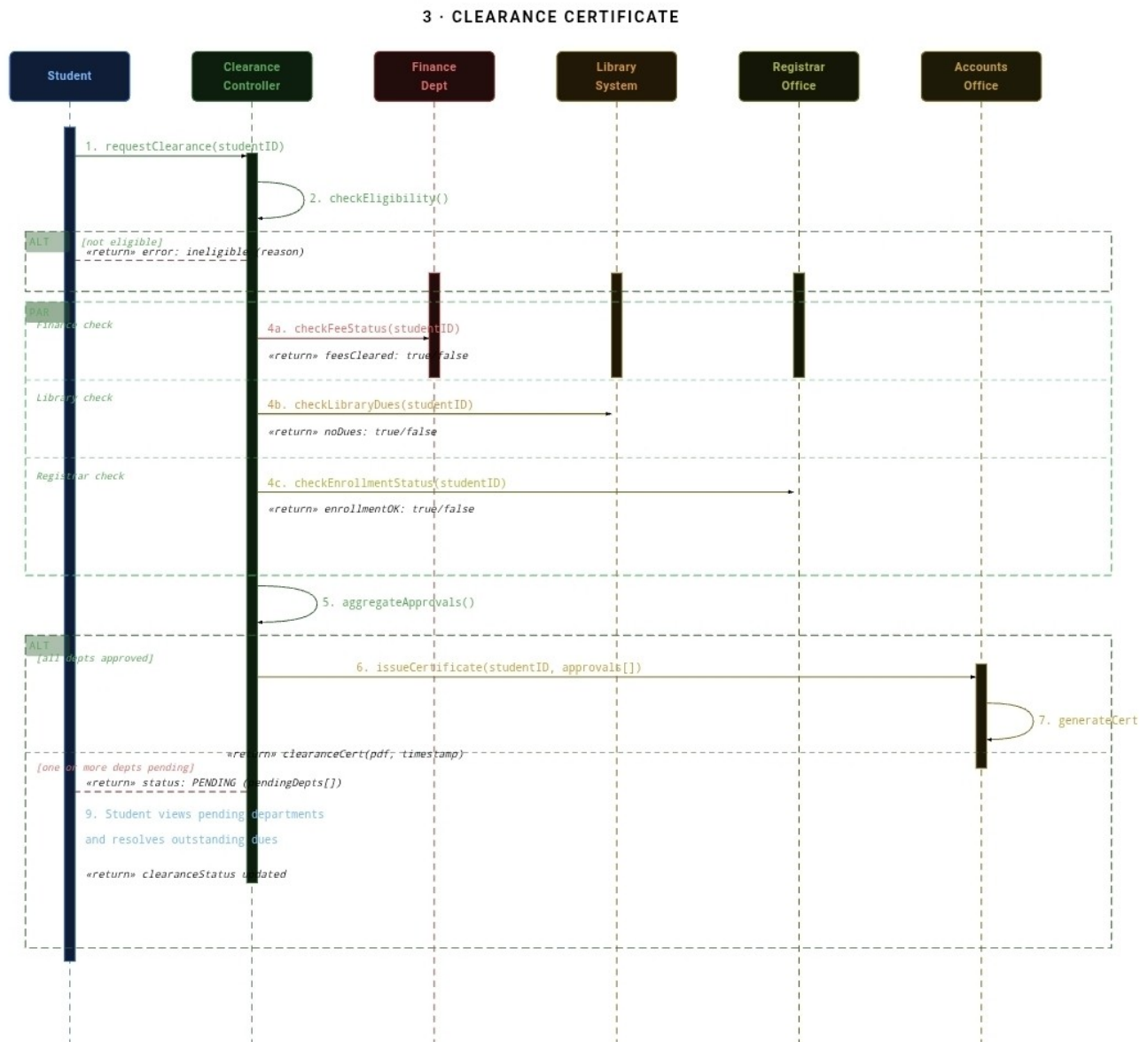
The Student calls viewChallan(studentID) and receives challan details. The Student calls initiatePayment(challanID, method). FeeController calls processPayment() on PaymentGateway — an ALT fragment handles payment success vs. failure. On failure, a LOOP fragment allows up to 3 retries. On success, FeeController calls verifyPayment(transactionRef) on AccountsOffice. AccountsOffice calls logAuditEntry() and generateReceipt(paymentDetails) on ReceiptService. The receipt is returned to the Student. AccountsOffice calls sendNotification() on NotificationService, which delivers a push notification to the Student.



2.4.3 Clearance Certificate Sequence

Objects: Student | ClearanceController | FinanceDept | LibrarySystem | RegistrarOffice | AccountsOffice

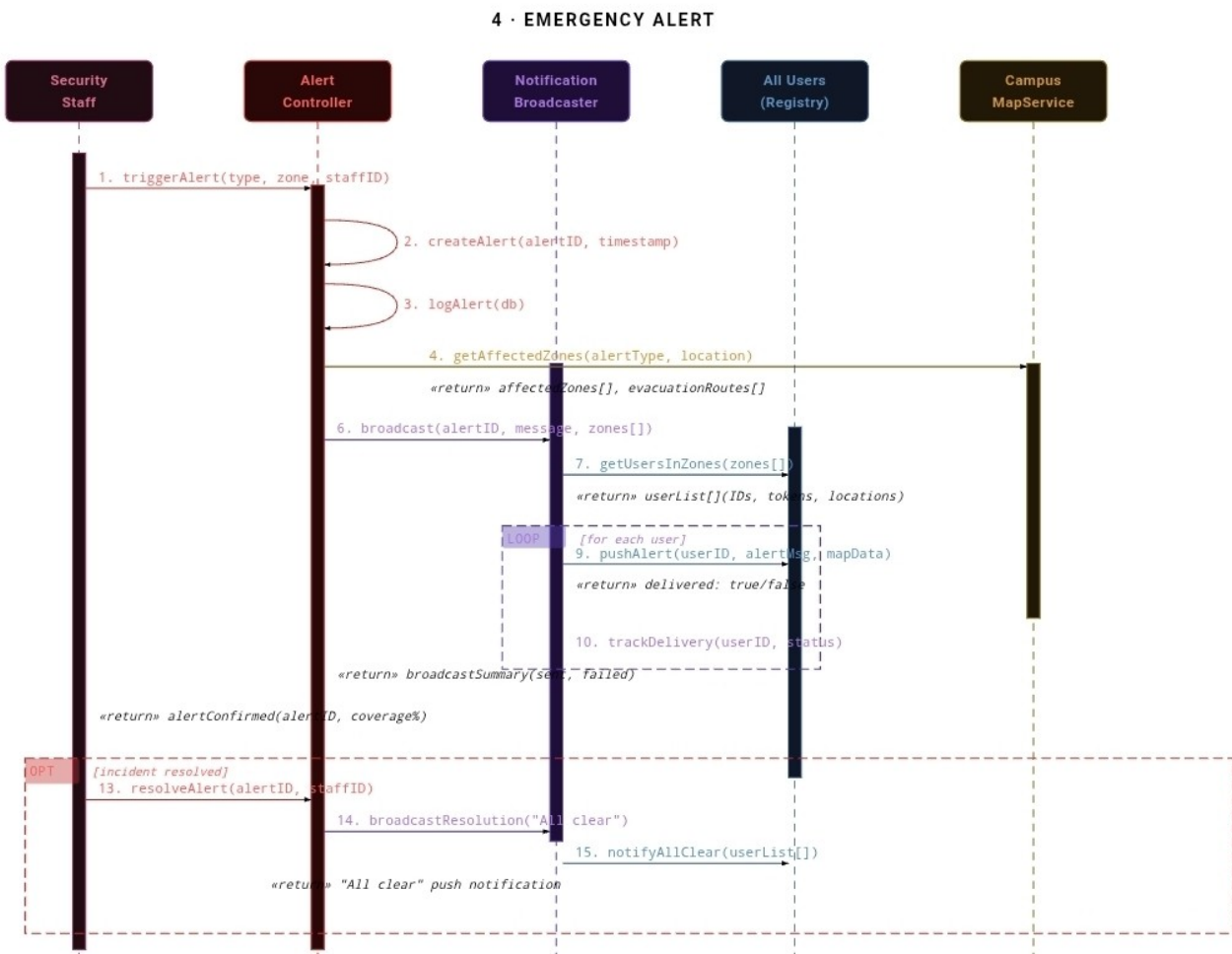
The Student calls requestClearance(studentID). ClearanceController calls checkEligibility() (self-call). An ALT fragment returns an error if ineligible. A PAR fragment sends simultaneous calls to Finance (checkFeeStatus), Library (checkLibraryDues), and Registrar (checkEnrollmentStatus). After all departments respond, ClearanceController calls aggregateApprovals(). An ALT fragment handles all-approved vs. pending cases. On approval, ClearanceController calls issueCertificate() on AccountsOffice, which calls generateCertificate(). The certificate PDF is returned to the Student portal.



2.4.4 Emergency Alert Sequence

Objects: SecurityStaff | AlertController | NotificationBroadcaster | AllUsers (Registry) | CampusMapService

SecurityStaff calls triggerAlert(type, zone, staffID). AlertController calls createAlert() and logAlert() (self-calls). AlertController calls getAffectedZones(alertType, location) on CampusMapService, receiving zones[] and evacuationRoutes[]. AlertController calls broadcast(alertID, message, zones[]) on NotificationBroadcaster. The Broadcaster calls getUsersInZones() on AllUsers Registry. A LOOP fragment sends pushAlert(userID, alertMsg, mapData) to each user, tracking delivery. The broadcast summary is returned to AlertController and confirmed to SecurityStaff. An OPT fragment handles incident resolution: SecurityStaff calls resolveAlert(), triggering broadcastResolution() and notifyAllClear() to all users.



2.5 Component Diagram

The Component Diagram presents the high-level software component structure of the Smart Campus Application, modeling the system as a set of modular, replaceable components that interact through well-defined interfaces. It illustrates the service-oriented architecture adopted by the system, showing how the client application, API gateway, business services, and external integrations are decoupled through interface-based communication.

2.5.1 Component Summary

The system is decomposed into ten primary components organized across four architectural layers:

Layer	Component	Type	Description
Client Layer	Client App	«component»	Mobile and web frontend applications providing the user interface
Gateway Layer	API Gateway	«component»	Central request router, load balancer, and SSL termination point
Gateway Layer	Auth Service	«component»	Authentication and authorization service validating JWT tokens
Business Layer	Academic Service	«component»	Handles attendance, courses, timetables, and QR code generation
Business Layer	Finance Service	«component»	Manages fee challans, payments, scholarships, and financial records
Business Layer	Security Service	«component»	Handles emergency alerts, gate logs, visitor management, and campus security
Business Layer	Notification Service	«component»	Central notification dispatcher for push, SMS, and email
Data Layer	Database	«component»	Primary PostgreSQL database with read/write and replication
External	Payment GW	«external»	Third-party payment gateway (HBL/JazzCash)
External	Messaging APIs	«external»	External messaging providers (FCM, Twilio, SendGrid)

2.5.2 Interfaces and Dependencies

The components communicate through ten explicitly modeled interfaces, enforcing loose coupling and enabling independent deployment of services:

Interface	Provider	Consumers	Purpose
IRestClient	Client App	API Gateway	RESTful communication from client to gateway
IRouting	API Gateway	Academic Service, Finance Service, Security Service, Auth Service	Request routing and service discovery
IAuth	Auth Service	Notification Service	Token validation and secure service-to-service authentication
IAttendance	Academic Service	Client App	Attendance data and QR code operations

Interface	Provider	Consumers	Purpose
IPayment	Finance Service	Client App	Payment initiation and challan retrieval
IAAlert	Security Service	API Gateway	Emergency alert triggering and security event propagation
INotify	Notification Service	Security Service, Finance Service	Internal notification dispatch between services
IDatabase	Database	Academic Service, Finance Service, Security Service	Persistent data storage and retrieval
IPayGW	Payment GW	Finance Service	External payment processing
IPush	Messaging APIs	Notification Service	Push notification, SMS, and email delivery

Key architectural decisions reflected in the component diagram:

The API Gateway acts as the single entry point for all client requests, routing them to the appropriate business service via the IRouting interface.

The Auth Service is isolated as a dedicated component, providing IAuth to the Notification Service and consuming IRouting from the gateway, ensuring centralized security policy enforcement.

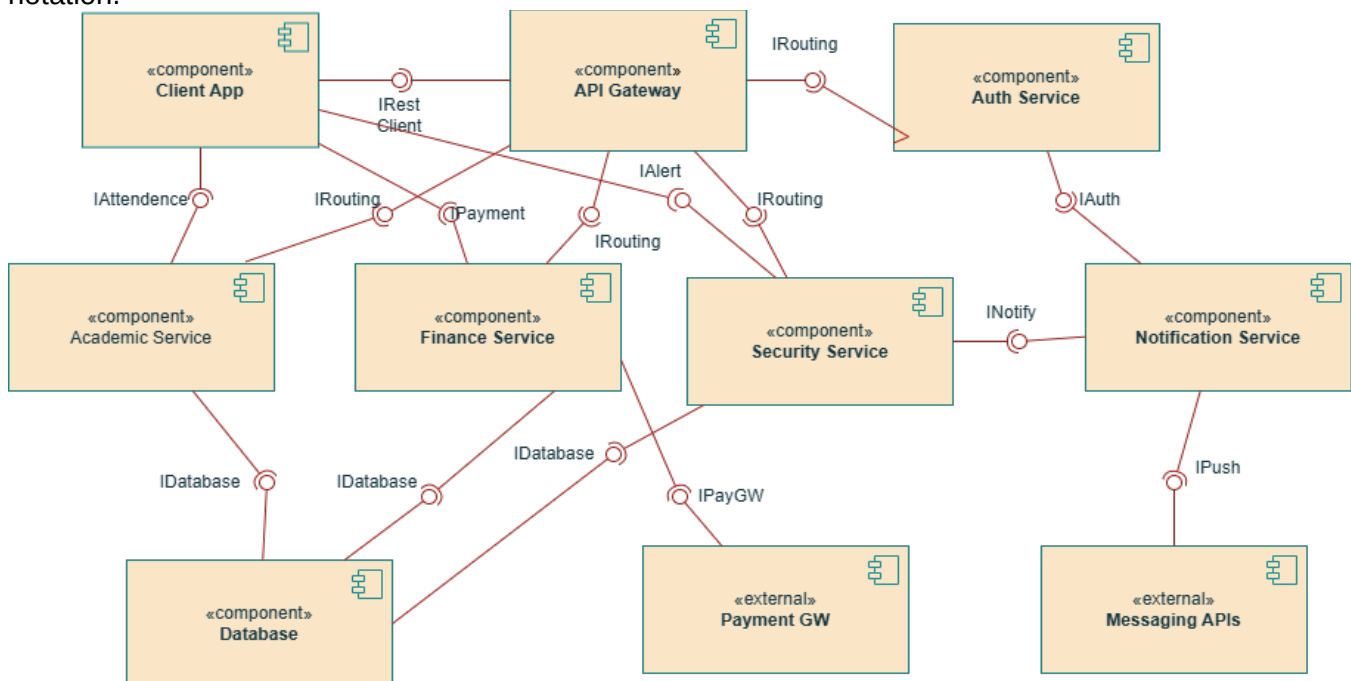
Business services (Academic, Finance, Security) are independent and communicate with the Database through a common IDatabase interface, allowing the data layer to evolve without impacting service logic.

The Notification Service acts as a cross-cutting concern, consumed by both Security Service (INotify) and Finance Service (indirectly via gateway routing), centralizing all outbound messaging.

External components (Payment GW and Messaging APIs) are stereotyped as «external» to clearly distinguish third-party dependencies from internal system components.

2.5.3 Component Diagram

The Component Diagram was developed using Draw.io and is presented below. All ten components, ten interfaces, and their provider/consumer relationships are shown with UML lollipop-and-socket notation.



2.6 Deployment Diagram

The Deployment Diagram illustrates the physical architecture of the Smart Campus Application, showing how software artifacts are distributed across hardware nodes, organized into security zones. The system is structured in five distinct zones arranged from public-facing to internal infrastructure.

2.6.1 Network Zones

Zone	Nodes	Purpose
Internet / Client Zone	Mobile Device, Admin Workstation, Security Terminal, QR Gate Scanner	Physical devices used by end users to access the system
DMZ	Network Firewall, Load Balancer, API Gateway Server	Secure perimeter layer providing traffic filtering, SSL termination, and request routing
Internal App Zone	App Server 1, App Server 2, Monitoring Server	Application servers hosting all business logic microservices in Docker containers
Data Zone	Primary DB Server, Replica DB Server, Redis Cache, File Storage Server, Audit Log DB	Persistence layer with high-availability database replication and caching
External Services	Payment GW, SMS Provider, Email Service, Push Notif (FCM), GPS/Maps API, Elasticsearch	Third-party services consumed by the application via HTTPS APIs

2.6.2 Key Artifacts and Technology Stack

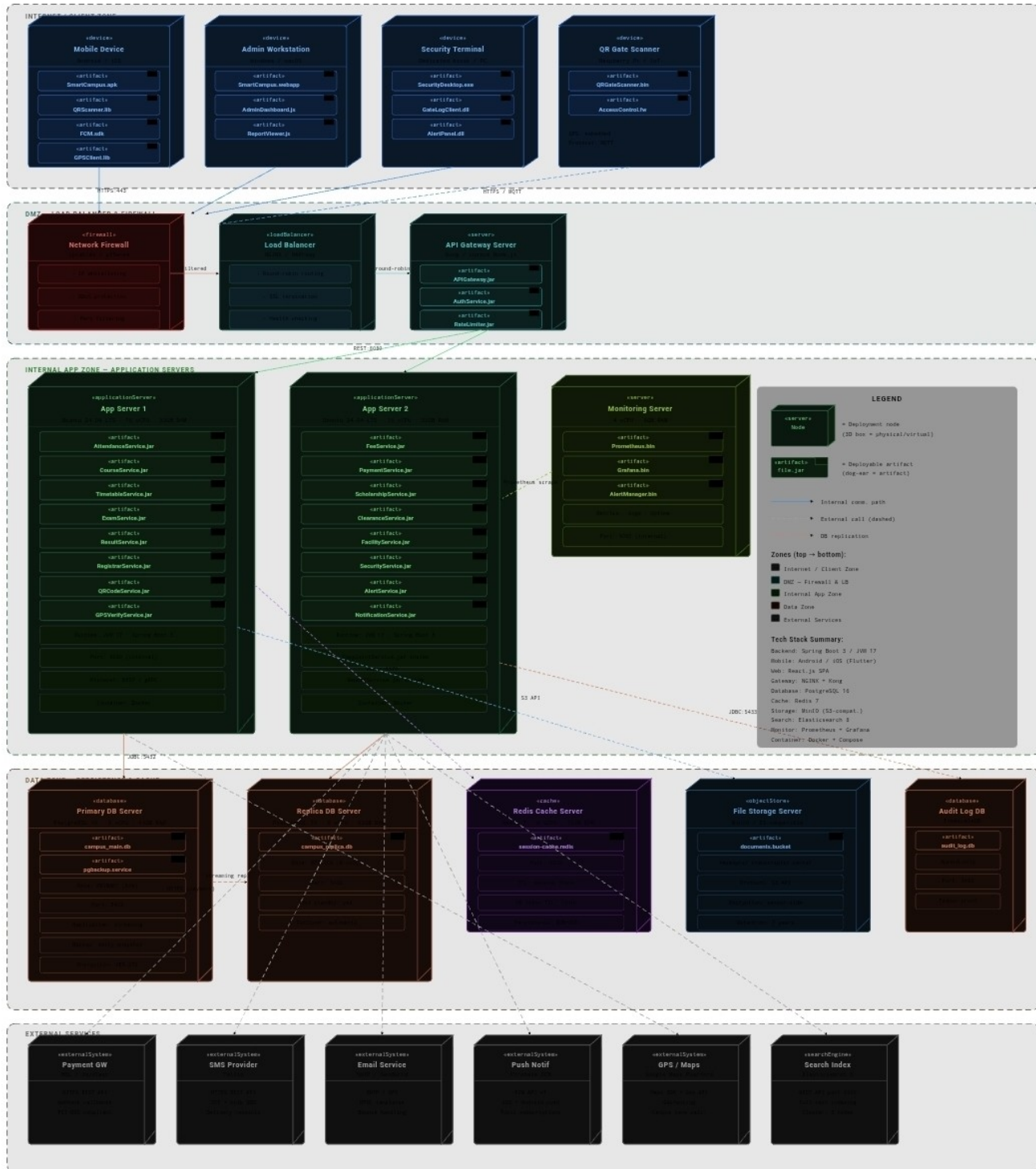
Layer	Technology	Artifact(s)
Backend	Spring Boot 3 / JVM 17	AttendanceService.jar, FeeService.jar, AlertService.jar, etc.
Mobile Client	Android / iOS (Flutter)	SmartCampus.apk, QRScanner.lib, FCM.sdk, GPSClient.lib
Web Client	React.js SPA	SmartCampus.webapp, AdminDashboard.js, ReportViewer.js
API Gateway	NGINX + Kong	APIGateway.jar, AuthService.jar, RateLimiter.jar
Database	PostgreSQL 16	campus_main.db (Primary R/W), campus_replica.db (Hot Standby)
Cache	Redis 7	session-cache.redis (TTL: 30min session / 10min QR token)
File Storage	MinIO (S3-compatible)	documents.bucket (receipts, transcripts, certificates)
Audit Logging	TimescaleDB	audit_log.db (append-only, tamper-proof)
Monitoring	Prometheus + Grafana	Prometheus.bin, Grafana.bin, AlertManager.bin
Containerization	Docker + Compose	All app server services containerized

2.6.3 Deployment Diagram

The Deployment Diagram presents all nodes as UML 3D boxes (physical/virtual nodes) with dog-ear artifacts inside each node. Communication paths are shown with protocol labels (HTTPS:443, REST:8080, JDBC:5432, etc.). External calls are shown as dashed lines.

SMART CAMPUS APPLICATION — DEPLOYMENT DIAGRAM

SDA Project · FAST University · 24K-0782 · 24K-0656



SMART CAMPUS APPLICATION – DEPLOYMENT DIAGRAM
SDA Project - FAST University - 24K-0782 - 24K-0656



Chapter 3: Architectural View (4+1 Model)

The 4+1 View Model provides a comprehensive description of the Smart Campus Application's architecture from five complementary perspectives. Each view addresses the concerns of a specific stakeholder group and is supported by diagrams produced in Chapter 2. Existing artifacts are mapped to each architectural view.

3.1 Logical View

The Logical View describes the functional decomposition of the system from the designer's and analyst's perspective. It addresses what the system does in terms of domain objects and their relationships.

Diagram reference: Class Diagram

The Smart Campus Application is organized into 7 functional packages (Users & Roles, Academic, Examination, Finance, Security, Communications, and Facilities & Clearance) plus an Enumerations package. The central abstraction is the User abstract class, from which Student, Teacher, and StaffMember (with Admin and SecurityStaff as concrete subclasses) are derived. Key domain relationships include:

- Students enroll in Courses and receive AttendanceRecords linked to QRCode objects generated by Teachers.
- FeeRecords are composed of FeeLineItems and settled through Payment objects, with optional Scholarship deductions.
- Clearance objects are composed of DeptApproval instances, one per approving department.
- EmergencyAlert objects are created by SecurityStaff and broadcast through the Notification subsystem.

The BCE classification further divides all classes into Boundary (20 classes), Control (10 classes), and Entity (28 classes) stereotypes, enforcing a strict layered communication pattern.

3.2 Process View

The Process View describes the system's runtime behavior, concurrency, synchronization, and control flow from the system integrator's perspective.

Diagram references: Activity Diagrams, Sequence Diagrams

The Smart Campus Application involves several concurrent processes:

- Parallel department checks in the Clearance Certificate workflow (PAR fragment in Sequence 3 and Activity Diagram 3) — Finance, Library, and Registrar checks execute simultaneously and are joined before certificate issuance.
- Fork/join parallel execution in the Fee Payment workflow — receipt generation and audit logging execute concurrently after payment verification.
- Emergency alert broadcasting uses a LOOP fragment to push alerts to all users in affected zones, with delivery tracking per user.

- QR code expiry management runs as a background process in the Redis cache layer with a 10-minute TTL.

3.3 Development View

The Development View describes the organization of the software in terms of modules, packages, and subsystems from the developer's perspective.

Diagram reference: Component Diagram, Class Diagram package structure

The system is organized into the following development modules, each corresponding to a bounded subsystem:

- Client Layer: MobileApp, WebApp, DesktopApp — three separate frontend clients sharing the same REST API.
- API Gateway Layer: APIGateway, AuthService, RateLimiter — centralized request routing, authentication, and rate limiting.
- Business Logic Layer: 10 service components organized into Academic, Finance, Examination, Security, Communications, and Facilities & Clearance modules, plus 5 shared services (GPSVerificationService, QRCodeService, AuditLogService, FileStorageService, SearchService).
- Data Access Layer: MainDatabase (PostgreSQL), CacheLayer (Redis), FileStorage (MinIO), SearchIndex (Elasticsearch), AuditLogDB (TimescaleDB).

All business logic services are packaged as Spring Boot JAR artifacts and deployed in Docker containers on application servers. The modular design allows individual services to be updated, scaled, or replaced independently.

3.4 Physical View

The Physical View describes the mapping of software artifacts onto hardware nodes and the network topology from the system engineer's perspective.

Diagram reference: Deployment Diagram

The physical deployment consists of five network zones:

- Internet/Client Zone hosts four device types: mobile devices (Android/iOS), admin workstations (web browser), security terminals (desktop application), and IoT QR gate scanners (Raspberry Pi with MQTT protocol).
- DMZ houses the Network Firewall (iptables/pfSense), Load Balancer (NGINX with round-robin routing), and API Gateway Server (Kong). All external traffic passes through this zone before reaching internal servers.
- Internal App Zone contains two horizontally-scaled application servers (Ubuntu 24.04, 16 vCPU, 32GB RAM, Docker) and a Monitoring Server (Prometheus + Grafana). All inter-service communication uses REST on port 8080.
- Data Zone houses the Primary PostgreSQL DB (port 5432, read/write) with streaming replication to a hot-standby Replica, Redis Cache (port 6379), MinIO object storage, and an append-only TimescaleDB audit log.

- External Services zone represents third-party systems accessed via HTTPS: HBL/JazzCash (payment), Twilio (SMS), SendGrid (email), Firebase FCM (push), Google Maps Platform (GPS/geofencing), and Elasticsearch (search index).

3.5 +1 Use Case View

The Use Case View serves as the unifying perspective that ties all other views together. It describes how the system's primary scenarios — represented as use cases — drive and validate the design decisions in the Logical, Process, Development, and Physical views.

Diagram reference: Use Case Diagram

The primary use cases and their cross-view significance are:

- Scan QR Attendance drives the QRCode and AttendanceRecord entity classes (Logical View), the GPS verification parallel process (Process View), the QRCodeService and GPSVerificationService components (Development View), and the QR Gate Scanner IoT device deployment (Physical View).
- Pay Fee drives the FeeRecord, Payment, and FeeLineItem entities (Logical), the parallel receipt-generation/audit-log fork (Process), the PaymentService and ScholarshipService components (Development), and the external payment gateway node (Physical).
- Request Clearance drives the Clearance and DeptApproval entities (Logical), the parallel department check PAR fragment (Process), the ClearanceService and ClearanceController components (Development), and the inter-server communication across the Internal App Zone (Physical).
- Trigger Emergency Alert drives the EmergencyAlert entity and Notification subsystem (Logical), the broadcast LOOP and OPT resolution fragment (Process), the AlertService and CampusMapService components (Development), and the push notification pipeline through Firebase FCM (Physical).

Chapter 4: References

- Object Management Group (OMG). (2017). Unified Modeling Language Specification, Version 2.5.1. Retrieved from <https://www.omg.org/spec/UML/2.5.1>
- Sparx Systems. (2024). Enterprise Architect — UML Modeling Tool. Retrieved from <https://sparxsystems.com>
- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.
- Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley.
- Spring Framework Documentation — Spring Boot 3 Reference Guide. Retrieved from <https://docs.spring.io/spring-boot>
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. Retrieved from <https://www.postgresql.org/docs/16/>
- Redis Ltd. (2024). Redis 7 Documentation. Retrieved from <https://redis.io/docs>
- Google Firebase Documentation — Firebase Cloud Messaging. Retrieved from <https://firebase.google.com/docs/cloud-messaging>
- NGINX Inc. (2024). NGINX Documentation. Retrieved from <https://nginx.org/en/docs/>
- Docker Inc. (2024). Docker Documentation. Retrieved from <https://docs.docker.com>
- Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- Rumbaugh, J., Jacobson, I., & Booch, G. (2004). The Unified Modeling Language Reference Manual (2nd ed.). Addison-Wesley.